Introduction aux analyses de survie

David Hajage

17 novembre 2026

Sommaire

Introduction **3**
En ligne ou via Rstudio ? . . . 3
Solutions . . . 3

1 Inspection de la base de données **5**

2 Description de la survie **10**
2.1 Construction du délai de survie . . . 10
2.2 Estimation de la survie globale par la méthode de Kaplan-Meïer . . . 13
2.3 Indicateurs . . . 18
2.3.1 Médiane de survie . . . 19
2.3.2 Suivi médian . . . 19

3 Comparaison de la survie de plusieurs groupes **21**
3.1 Survie globale par groupe . . . 21
3.2 Comparaison de la survie globale . . . 24

4 Modèle de Cox **27**
4.1 Modèle de Cox univariable . . . 27
4.2 Hypothèse des hasards proportionnels . . . 30
4.3 Modèle de Cox multivariable . . . 32

5 Risques compétitifs **35**
5.1 Délai jusqu'à récidive, survie sans récidive . . . 35
5.2 Délai jusqu'à récidive ou décès sans récidive en risques compétitifs . . . 37
5.3 Incidence cumulée des évènements . . . 38
5.3.1 Approche 'naïve' . . . 38
5.3.2 Approche correcte . . . 39
5.4 Modèle de Cox cause-spécifique . . . 40
5.4.1 En deux commandes . . . 40
5.4.2 En une seule commande . . . 41
5.5 Modèle de Fine et Gray . . . 41
5.6 Comparaison des modèles causes-spécifiques et de sous-répartition . . . 42

Conclusion **43**

Introduction

Bienvenue dans cette introduction aux analyses de survie ! Dans ce tutoriel, vous allez apprendre les bases de l'utilisation du package `survival`, le principal package dédié aux analyses de survie avec le logiciel R.

En ligne ou via Rstudio ?

Vous pouvez suivre ce TP en ligne depuis cette page en utilisant les blocs de code interactifs mis à disposition tout au long du TP, ou bien utiliser une session locale de Rstudio. Si vous préférez utiliser Rstudio, il faudra télécharger la base de données `data_bone.csv` (via les liens en bas de cette page), et l'importer via les commandes suivantes :

```
## Importation de la base de données au format csv
## Attention de modifier le chemin vers le dossier de téléchargement !
bone <- read.csv2("chemin/vers/data_bone.csv")
## Mise des dates au format "Date"
bone$dat_sel <- as.Date(bone$dat_sel, format = "%d/%m/%Y")
bone$date_dc <- as.Date(bone$date_dc, format = "%d/%m/%Y")
bone$ddn <- as.Date(bone$ddn, format = "%d/%m/%Y")
bone$date_rec <- as.Date(bone$date_rec, format = "%d/%m/%Y")
bone$date_fn <- as.Date(bone$date_fn, format = "%d/%m/%Y")
bone$date_cc <- as.Date(bone$date_cc, format = "%d/%m/%Y")
bone$date_ca <- as.Date(bone$date_ca, format = "%d/%m/%Y")
```

Si vous faites ce TP en ligne, voici ci-dessous un bloc de code interactif :

Tapez la commande `2+2` ci-dessus. Pour exécuter le code, deux solutions :

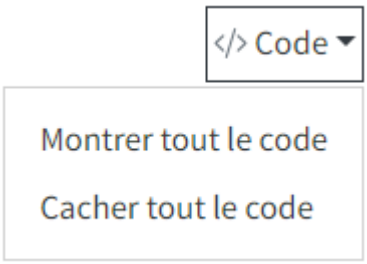
— Cliquer sur 

(cela exécutera l'ensemble du code contenu dans le bloc de code interactif)
— Utiliser le raccourci clavier "Ctrl+Enter" (comme dans Rstudio, ce raccourci exécute le code ligne après ligne quand rien n'est sélectionné en surbrillance, ou alors le code que vous avez préalablement sélectionné en surbrillance avec votre souris).

Solutions

Ce TP peut être réalisé de manière autonome et à votre rythme (vous pourrez le terminer chez vous si la séance a été trop courte). A chaque étape, un code de solution est proposé : pour

le voir, il suffit de cliquer sur “Montrez moi comment faire !”. Il est possible de démasquer l'ensemble du code de solution d'une page en cliquant sur “</> Code” puis “Montrer tout le code”.



Ce bouton est disponible **en haut de chaque page web**, à droite du titre. Ne restez pas bloqué trop longtemps, et consultez rapidement ces solutions : le TP est long, les approches statistiques mises en oeuvre sont parfois complexes, le but du TP est d'en comprendre le fonctionnement et d'en voir un exemple d'application. Lisez attentivement les tâches à réaliser (écrites en *italique*), inspectez les variables créées par le code de solution, et lisez les commentaires.

Pour passer à la suite, appuyez sur le lien en bas à droite de cette page (ou sur la partie suivante à gauche) !

1 Inspection de la base de données

Si vous faites le TD avec Rstudio, la base de données doit avoir été importée avec les commandes suivantes :

```
bone <- read.csv2("chemin/vers/data_bone.csv")
bone$dat_sel <- as.Date(bone$dat_sel, format = "%d/%m/%Y")
bone$date_dc <- as.Date(bone$date_dc, format = "%d/%m/%Y")
bone$ddn <- as.Date(bone$ddn, format = "%d/%m/%Y")
bone$date_rec <- as.Date(bone$date_rec, format = "%d/%m/%Y")
bone$date_fn <- as.Date(bone$date_fn, format = "%d/%m/%Y")
bone$date_cc <- as.Date(bone$date_cc, format = "%d/%m/%Y")
bone$date_ca <- as.Date(bone$date_ca, format = "%d/%m/%Y")
```

La base de données utilisée dans les démonstrations et les exercices provient d'une étude observationnelle chez des patients atteints d'une leucémie.

Vous pouvez écrire et lancer du code dans la boite ci-dessous. Pour le moment, n'ajoutez aucun code ci-dessous, cliquez juste sur le bouton Run Code.

```
# Ceci est un commentaire
# La base de données a déjà été importée à l'aide de la commande
# read.csv2("data_bone.csv")
# Vous n'avez rien besoin d'écrire ici, appuyez juste sur le bouton 'Run Code'
# ou sur le raccourci clavier Ctrl+Enter

str(bone)
```

Cette commande affiche un résumé basique de la base de données (ou plutôt la data.frame) `bone`. On sait qu'il y a 137 lignes (individus) et 24 colonnes (variables). Si on ne s'intéresse qu'aux noms des variables, on peut utiliser la fonction `names()`.

Essayez d'utiliser la fonction `names()`, avec le nom de la data.frame comme seul argument. Si vous êtes coincé · e, cliquez sur 'Montrez moi comment faire !' juste en dessous de la boite interactive (il y en aura une à chaque fois).

Correction :

```
# Copier-collez cette solution ci-dessus :
names(bone)
```

```
 [1] "num"      "leuc"     "etat"     "rec"      "ca"       "cc"
 [7] "fn"       "age.pat"  "age.don"  "sexe.pat" "sexe.don" "cmv.pat"
[13] "cmv.don"  "delatt"   "fab"      "hop"      "mtx"      "dat_sel"
[19] "ddn"      "date_dc"  "date_rec" "date_ca"  "date_cc"  "date_fn"
```

Ces deux fonctions sont utiles pour savoir le contenu de vos données, mais c'est souvent utile de regarder un échantillon de quelques lignes. Essayez d'utiliser la fonction `head()`.

Utilisez la fonction `head()`, à nouveau avec le nom de la data.frame comme seul argument.

Correction :

```
head(bone)
```

```
  num leuc etat rec ca cc fn age.pat age.don sexe.pat sexe.don cmv.pat cmv.don
1   1    1    0   0  1  1  1      26      33        1        0       1       1
2   2    1    0   0  0  1  1      21      37        1        1       0       0
3   3    1    0   0  0  1  1      26      35        1        1       1       0
4   4    1    0   0  1  1  1      17      21        0        1       0       0
5   5    1    0   0  0  1  1      32      36        1        1       1       1
6   6    1    0   0  0  1  1      22      31        1        1       1       1
  delatt fab hop mtx    dat_sel        ddn date_dc date_rec    date_ca
1     98   0   1   0 2011-10-05 2017-06-16    <NA>     <NA> 2011-12-11
2   1720   0   1   0 2011-10-09 2016-02-27    <NA>     <NA>       <NA>
3    127   0   1   0 2011-10-10 2015-11-14    <NA>     <NA>       <NA>
4    168   0   1   0 2011-10-10 2015-10-11    <NA>     <NA> 2011-12-19
5     93   0   1   0 2011-10-11 2015-09-13    <NA>     <NA>       <NA>
6   2187   0   1   0 2011-10-11 2015-07-19    <NA>     <NA>       <NA>
     date_cc    date_fn
1 2012-02-03 2011-10-18
2       <NA> 2011-10-27
3       <NA> 2011-10-22
4       <NA> 2011-10-23
5       <NA> 2011-10-23
6       <NA> 2011-10-23
```

La fonction `head()` vous affichera par défaut les 6 premières lignes de votre data.frame. Mais on peut ajouter un **argument supplémentaire** à cette fonction pour préciser le nombre de lignes souhaitées. Dans R, beaucoup d'arguments sont optionnels (c'est-à-dire qu'ils ont une valeur par défaut), et ne sont pas nécessaires pour que la fonction… fonctionne, mais préciser une autre valeur que celle par défaut permet d'être plus spécifique.

Ajoutez `n = 10` comme second argument.

```
head(bone)
```

Correction :

```
# Copier-collez cette solution ci-dessus :
head(bone, n = 10)
```

```
   num leuc etat rec ca cc fn age.pat age.don sexe.pat sexe.don cmv.pat cmv.don
1    1    1    0   0  1  1  1      26      33        1        0       1       1
2    2    1    0   0  0  1  1      21      37        1        1       0       0
3    3    1    0   0  0  1  1      26      35        1        1       1       0
4    4    1    0   0  1  1  1      17      21        0        1       0       0
5    5    1    0   0  0  1  1      32      36        1        1       1       1
6    6    1    0   0  0  1  1      22      31        1        1       1       1
7    7    1    0   0  0  1  1      20      17        1        0       1       1
8    8    1    0   0  1  1  1      22      24        1        0       0       0
9    9    1    0   0  0  0  1      18      21        0        1       0       0
10  10    1    0   0  0  1  1      24      40        1        1       0       1
   delatt fab hop mtx    dat_sel        ddn date_dc date_rec    date_ca
1      98   0   1   0 2011-10-05 2017-06-16    <NA>     <NA> 2011-12-11
2    1720   0   1   0 2011-10-09 2016-02-27    <NA>     <NA>       <NA>
3     127   0   1   0 2011-10-10 2015-11-14    <NA>     <NA>       <NA>
4     168   0   1   0 2011-10-10 2015-10-11    <NA>     <NA> 2011-12-19
5      93   0   1   0 2011-10-11 2015-09-13    <NA>     <NA>       <NA>
6    2187   0   1   0 2011-10-11 2015-07-19    <NA>     <NA>       <NA>
7    1006   0   1   0 2011-10-16 2015-06-07    <NA>     <NA>       <NA>
8    1319   0   1   0 2011-10-10 2014-07-02    <NA>     <NA> 2011-12-21
9     208   0   1   0 2011-10-13 2012-05-26    <NA>     <NA>       <NA>
10    174   0   3   1 2011-10-17 2015-01-28    <NA>     <NA>       <NA>
      date_cc    date_fn
1  2012-02-03 2011-10-18
2        <NA> 2011-10-27
3        <NA> 2011-10-22
4        <NA> 2011-10-23
5        <NA> 2011-10-23
6        <NA> 2011-10-23
7        <NA> 2011-11-02
8        <NA> 2011-10-22
9        <NA> 2011-10-23
10       <NA> 2011-11-15
```

`str()`, `names()` et `head()` nous ont permis d'avoir un premier regard sur la base de données : celle-ci est constituée d'une ligne par individu, et de différentes colonnes. Le nom des colonnes n'est pas toujours suffisant pour comprendre son contenu. Le tableau ci-dessous est un dictionnaire de variables sommaire :

Variable	Label
num	Numéro d'identification
leuc	Leucémie, avec 3 types
	1 : leucémie myéloïde aigue B
	2 : leucémie myéloïde aigue C
	3 : leucémie aigue lymphoblastique
etat	Etat du sujet aux dernières nouvelles
	0 : vivant
	1 : décédé
rec	Récidive oui/non
	0 : pas de récidive
	1 : récidive
ca	Complication aigue
	0 : pas de complication
	1 : complication aigue
cc	complication chronique
	0 : pas de complication
	1 : complication chronique
fn	Fin de la neutropénie
	0 : neutropénie en cours aux dernières nouvelles
	1 : fin de la neutropénie.
age.pat	âge du patient (en année)
age.don	âge du donneur (en année)
sexe.pat	sexe du patient
	0 : Femme
	1 : Homme
sexe.don	sexe du donneur
	0 : Femme
	1 : Homme
cmv.pat	Infection au cytomégalovirus chez le patient
	0 : pas d'infection
	1 : infection
cmv.don	Infection au cytomégalovirus chez le donneur
	0 : pas d'infection
	1 : infection
delatt	Délai avant greffe (en jours)
fab	Classification des leucémies (Franco-Americain-Britannique)
	0 : Grade 4 ou 5 ou LMA
	1 : Autres
hop	Hopital de prise en charge
	1 : Ohio
	2 : Alfred
	3 Saint Vincent
	4 Hahnemann
mtx	Methotrexate
	0 : pas de MTX
	1 : MTX

Variable	Label
dat_sel	Date d'inclusion
ddn	Date de dernières nouvelles
date_dc	Date de décès
date_rec	Date de récidive
date_ca	Date de complication aigue
date_cc	Date de complication chronique
date_fn	Date de fin de neutropénie

Cette introduction est maintenant terminée !

Conseils importants avant de passer à la suite

- *Ne restez pas bloqué longtemps.* Prenez une minute pour bien relire les instructions, testez vos commandes, ce n'est pas grave de se tromper ou de ne pas trouver. Un bouton "Montrez moi comment faire !" est *toujours* disponible, n'hésitez pas à l'utiliser après 5 min de galère (notamment quand je vous demande de créer des nouvelles variables) : le but est de voir en pratique les notions abordées en cours, pas de devenir programmeur R.
- *De l'aide est disponible :*
  - Taper `?head` vous affichera la page d'aide correspondante, comme vous le faites habituellement dans Rstudio.
  - Le jour du TP, lever la main téléportera automatiquement l'enseignant auprès de vous.

- *Il n'y a pas toujours qu'une seule manière de faire.* La correction proposée à chaque question n'est qu'une proposition, en particulier quand on vous demande de créer une nouvelle variable ou de sélectionner des individus. Tout le monde a sa propre manière de programmer certaines choses.

Pour passer à la suite, appuyez sur le lien en bas à droite de cette page (ou sur la partie suivante à gauche) !

2 Description de la survie

Si vous faites le TD avec Rstudio, la base de données doit avoir été importée avec les commandes suivantes :

```
bone <- read.csv2("chemin/vers/data_bone.csv")
bone$dat_sel <- as.Date(bone$dat_sel, format = "%d/%m/%Y")
bone$date_dc <- as.Date(bone$date_dc, format = "%d/%m/%Y")
bone$ddn <- as.Date(bone$ddn, format = "%d/%m/%Y")
bone$date_rec <- as.Date(bone$date_rec, format = "%d/%m/%Y")
bone$date_fn <- as.Date(bone$date_fn, format = "%d/%m/%Y")
bone$date_cc <- as.Date(bone$date_cc, format = "%d/%m/%Y")
bone$date_ca <- as.Date(bone$date_ca, format = "%d/%m/%Y")
```

2.1 Construction du délai de survie

La base de données `bone` est à nouveau déjà chargée. Dans cette partie, nous allons nous intéresser à la survie globale, c'est à dire au délai jusqu'au décès des patients.

La base ne met pas à disposition le délai jusqu'au décès du patient : ce délai doit être créé à partir des informations disponibles :

- `dat_sel` : la date d'inclusion du patient = la date d'origine
- `ddn` : la date de dernière nouvelle du patient
- `date_dc` : la date de décès
- `etat` : l'état du patient aux dernières nouvelles

Par nature, la date de décès d'un patient met fin à son suivi. L'information contenue dans `date_dc` et `ddn` est donc en partie redondante : pour les patients décédés, la date de dernière nouvelle et la date de décès devraient être identiques.

Evaluez la cohérence du contenu des variables ***`date_dc`*** *et* ***`ddn`*** *pour les patients décédés, en vérifiant que les colonnes* ***`date_dc`*** *et* ***`ddn`*** *ont la même valeur chez eux.*

Correction :

```
## Avec les commandes R de base

bone_dc <- bone[bone$etat == 1, c("date_dc", "ddn")] ## Sélection des patients
                                                     ## décédés et des colonnes
                                                     ## "date_dc" et "ddn"
head(bone_dc) ## Premières lignes
```

```r
tail(bone_dc) ## Dernières lignes
table(bone_dc$date_dc == bone_dc$ddn, useNA = "ifany")
## Les dates de ces deux colonnes sont toutes identiques chez les individus décédés

## Même chose avec dplyr
# library(dplyr)
bone %>%
  filter(etat == 1) %>%
  mutate(same_date = date_dc == ddn) %>%
  count(same_date)
```

```
      date_dc        ddn
15 2012-12-08 2012-12-08
16 2012-12-15 2012-12-15
17 2012-08-01 2012-08-01
18 2012-04-07 2012-04-07
19 2013-11-28 2013-11-28
20 2012-04-20 2012-04-20
```

```
       date_dc        ddn
132 2012-11-05 2012-11-05
133 2013-08-13 2013-08-13
134 2012-09-27 2012-09-27
135 2012-11-10 2012-11-10
136 2013-01-05 2013-01-05
137 2013-03-31 2013-03-31
```

```
TRUE
  81
```

```
  same_date  n
1      TRUE 81
```

Quel est le contenu de la variable ***`date_dc`*** *chez les patients vivants aux dernières nouvelles ?*

Correction :

```r
bone_vv <- bone[bone$etat == 0, c("date_dc", "ddn")] ## Sélection des patients
                                                     ## vivants et des colonnes
                                                     ## "date_dc" et "ddn"
head(bone_vv)
tail(bone_vv)
table(bone_vv$date_dc, useNA = "ifany") ## Il n'y a que des NA. C'est normal !
```

```r
## Même chose avec dplyr
# library(dplyr)
bone %>%
  filter(etat == 0) %>%
  count(date_dc)
```

```
  date_dc        ddn
1    <NA> 2017-06-16
2    <NA> 2016-02-27
3    <NA> 2015-11-14
4    <NA> 2015-10-11
5    <NA> 2015-09-13
6    <NA> 2015-07-19
```

```
    date_dc        ddn
98     <NA> 2015-08-30
99     <NA> 2016-09-11
100    <NA> 2017-07-27
101    <NA> 2015-12-04
102    <NA> 2015-04-18
103    <NA> 2014-07-19
```

```
<NA>
  56
```

```
  date_dc  n
1    <NA> 56
```

Le délai à calculer est :

- la différence entre la date de décès et la date d'origine pour les patients décédés
- la différence entre la date de dernière nouvelle et la date d'origine pour les patients vivants

Comme la date de dernière nouvelle et la date de décès sont identiques en cas de décès, nous n'avons pas besoin de `date_dc` pour ce calcul.

En utilisant la fonction ***`difftime()`****, calculez le délai de suivi de chaque patient en jours, et stockez le dans la base de données dans une variable nommée* ***`delsv`****. La fonction a trois arguments :* ***`time1`*** *(la première date),* ***`time2`*** *(la seconde date), et* ***`units`*** *(utilisez "days").*

Correction :

```r
bone$delsv <- difftime(bone$ddn, bone$dat_sel, units = "days")
head(bone[, c("dat_sel", "ddn", "delsv", "etat")])
tail(bone[, c("dat_sel", "ddn", "delsv", "etat")])

## Même chose avec dplyr
# library(dplyr)
bone <- bone %>%
  mutate(delsv = difftime(ddn, dat_sel, units = "days"))
```

```
     dat_sel        ddn     delsv etat
1 2011-10-05 2017-06-16 2081 days    0
2 2011-10-09 2016-02-27 1602 days    0
3 2011-10-10 2015-11-14 1496 days    0
4 2011-10-10 2015-10-11 1462 days    0
5 2011-10-11 2015-09-13 1433 days    0
6 2011-10-11 2015-07-19 1377 days    0
```

```
       dat_sel        ddn    delsv etat
132 2012-07-23 2012-11-05 105 days    1
133 2012-07-17 2013-08-13 392 days    1
134 2012-07-26 2012-09-27  63 days    1
135 2012-08-05 2012-11-10  97 days    1
136 2012-08-05 2013-01-05 153 days    1
137 2012-04-02 2013-03-31 363 days    1
```

2.2 Estimation de la survie globale par la méthode de Kaplan-Meïer

Nous avons tous les ingrédients nécessaires pour estimer la survie globale dans cet échantillon de patients.

Le package permettant de réaliser des analyses de données censurées dans R s'appelle `survival`. Il se charge avec la commande `library(survival)` (le package est déjà chargé dans ce TP, taper cette commande est donc inutile, mais il faudra le faire dans vos analyses).

La fonction qui permet de réaliser l'estimation de Kaplan-Meier s'appelle `survfit()`. Les paramètres de cette fonction sont une formule, puis la data.frame qui contient les données.

Lancez la commande ci-dessous.

```r
# Vous n'avez rien besoin d'écrire ici, appuyez juste sur le bouton 'Run Code'
# ou sur le raccourci clavier Ctrl+Enter
survfit(Surv(delsv, etat) ~ 1, data = bone)
```

`Surv(delsv, etat)` permet d'indiquer à la fonction le nom des variables contenant respectivement le délai de suivi, et le statut vis-à-vis de l'évènement à la fin de ce suivi. La partie `~ 1` est un peu particulière : elle ne sert pas à grand chose ici, mais modifier cette partie permettra plus tard d'estimer la survie globale selon certains sous-groupes.

Cette commande affiche quelques résultats : l'effectif `n`, le nombre de décès observés `events`, la médiane de survie `median`, ainsi que l'intervalle de confiance à 95% autour de cette médiane. Il s'agit bien sûr d'un affichage partiel… Il manque les probabilités de survie !

Stockez le résultat de la commande `survfit()` dans un objet appelé `fit_km`, puis appliquez la fonction `summary()` à cet objet.

Correction :

```
fit_km <- survfit(Surv(delsv, etat) ~ 1, data = bone)
summary(fit_km)
```

```
Call: survfit(formula = Surv(delsv, etat) ~ 1, data = bone)

 time n.risk n.event survival std.err lower 95% CI upper 95% CI
    1    137       1    0.993 0.00727        0.979        1.000
    2    136       1    0.985 0.01025        0.966        1.000
   10    135       1    0.978 0.01250        0.954        1.000
   16    134       1    0.971 0.01438        0.943        0.999
   35    133       1    0.964 0.01602        0.933        0.995
   48    132       1    0.956 0.01748        0.923        0.991
   53    131       1    0.949 0.01881        0.913        0.987
   62    130       1    0.942 0.02003        0.903        0.982
   63    129       1    0.934 0.02117        0.894        0.977
   73    128       1    0.927 0.02222        0.884        0.972
   74    127       1    0.920 0.02322        0.875        0.966
   79    126       1    0.912 0.02415        0.866        0.961
   80    125       2    0.898 0.02588        0.848        0.950
   86    123       1    0.891 0.02668        0.840        0.944
   93    122       1    0.883 0.02744        0.831        0.939
   97    121       1    0.876 0.02817        0.822        0.933
  105    120       3    0.854 0.03017        0.797        0.915
  107    117       1    0.847 0.03078        0.788        0.909
  110    116       1    0.839 0.03137        0.780        0.903
  121    115       1    0.832 0.03193        0.772        0.897
  122    114       2    0.818 0.03300        0.755        0.885
  128    112       1    0.810 0.03350        0.747        0.879
  129    111       1    0.803 0.03399        0.739        0.872
  153    110       1    0.796 0.03445        0.731        0.866
  156    109       1    0.788 0.03490        0.723        0.860
  162    108       2    0.774 0.03575        0.707        0.847
```

164	106	1	0.766	0.03615	0.699	0.841
168	105	1	0.759	0.03653	0.691	0.834
172	104	1	0.752	0.03690	0.683	0.828
183	103	1	0.745	0.03726	0.675	0.821
194	102	1	0.737	0.03760	0.667	0.815
195	101	1	0.730	0.03793	0.659	0.808
222	100	1	0.723	0.03825	0.651	0.802
243	98	1	0.715	0.03856	0.644	0.795
248	97	1	0.708	0.03886	0.636	0.788
262	96	2	0.693	0.03943	0.620	0.775
265	94	1	0.686	0.03969	0.612	0.768
269	93	1	0.678	0.03995	0.604	0.761
276	92	1	0.671	0.04019	0.597	0.755
288	91	1	0.664	0.04042	0.589	0.748
318	90	1	0.656	0.04063	0.581	0.741
341	89	1	0.649	0.04084	0.574	0.734
350	88	1	0.642	0.04104	0.566	0.727
363	87	1	0.634	0.04122	0.558	0.720
371	86	1	0.627	0.04140	0.551	0.713
390	85	1	0.619	0.04156	0.543	0.706
392	84	1	0.612	0.04172	0.535	0.699
393	83	1	0.605	0.04186	0.528	0.693
414	82	1	0.597	0.04199	0.520	0.686
417	81	1	0.590	0.04212	0.513	0.679
418	80	1	0.583	0.04223	0.505	0.671
431	79	1	0.575	0.04234	0.498	0.664
466	78	1	0.568	0.04243	0.490	0.657
469	77	1	0.560	0.04252	0.483	0.650
481	76	1	0.553	0.04259	0.476	0.643
487	75	1	0.546	0.04266	0.468	0.636
491	74	1	0.538	0.04271	0.461	0.629
515	73	1	0.531	0.04276	0.453	0.622
522	72	1	0.524	0.04280	0.446	0.615
526	71	1	0.516	0.04282	0.439	0.607
547	69	1	0.509	0.04285	0.431	0.600
583	68	1	0.501	0.04287	0.424	0.593
641	67	1	0.494	0.04288	0.416	0.585
653	66	1	0.486	0.04288	0.409	0.578
677	65	1	0.479	0.04286	0.402	0.571
704	64	1	0.471	0.04284	0.394	0.563
716	63	1	0.464	0.04281	0.387	0.556
732	62	1	0.456	0.04277	0.380	0.548
781	61	1	0.449	0.04272	0.372	0.541
1063	52	1	0.440	0.04276	0.364	0.533
1074	51	1	0.432	0.04278	0.355	0.524
1156	48	1	0.423	0.04282	0.346	0.515
1279	42	1	0.413	0.04297	0.336	0.506
1298	41	1	0.402	0.04308	0.326	0.496

```
 2201      9        1    0.358 0.05696        0.262        0.489
```

R affiche maintenant les probabilités de survie `survival` (avec leur IC95%) à chaque temps de décès observé dans l'échantillon (`time`). La colonne `n.risk` est le nombre de sujets encore à risque au cours du temps. `n.event` vous indique le nombre de décès observés dans chaque intervalle de temps. Il est souvent utile de demander à R de n'afficher que certains temps précis. Pour cela, il suffit d'utiliser l'option `times` dans la commande `summary`.

Ajoutez `times = c(0, 365, 730)` à la commande `summary`.

```r
summary(fit_km)
```

Correction :

```r
summary(fit_km, times = c(0, 365, 730))
```

```
Call: survfit(formula = Surv(delsv, etat) ~ 1, data = bone)

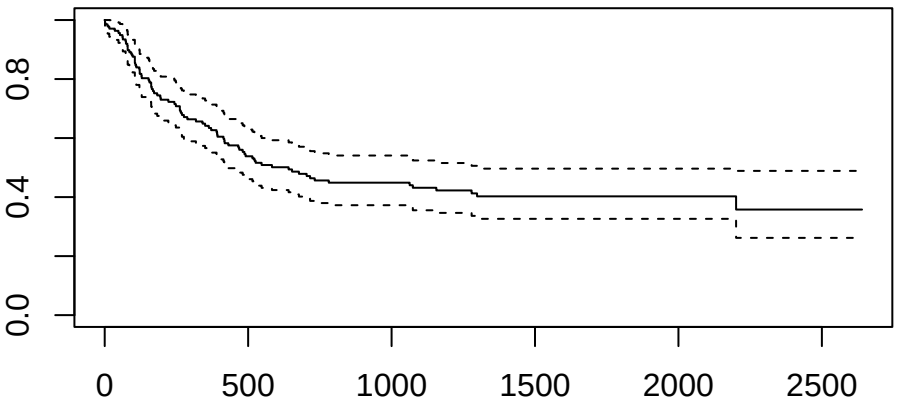
 time n.risk n.event survival std.err lower 95% CI upper 95% CI
    0    137       0    1.000  0.0000        1.000        1.000
  365     86      50    0.634  0.0412        0.558        0.720
  730     62      23    0.464  0.0428        0.387        0.556
```

`summary()` n'est pas la seule fonction applicable aux résultats produits par `survfit()`. La fonction `plot()` permet de tracer la courbe de Kaplan-Meier.

Utilisez la fonction `plot()` sur l'objet `fit_km`.

Correction :

```r
plot(fit_km)
```

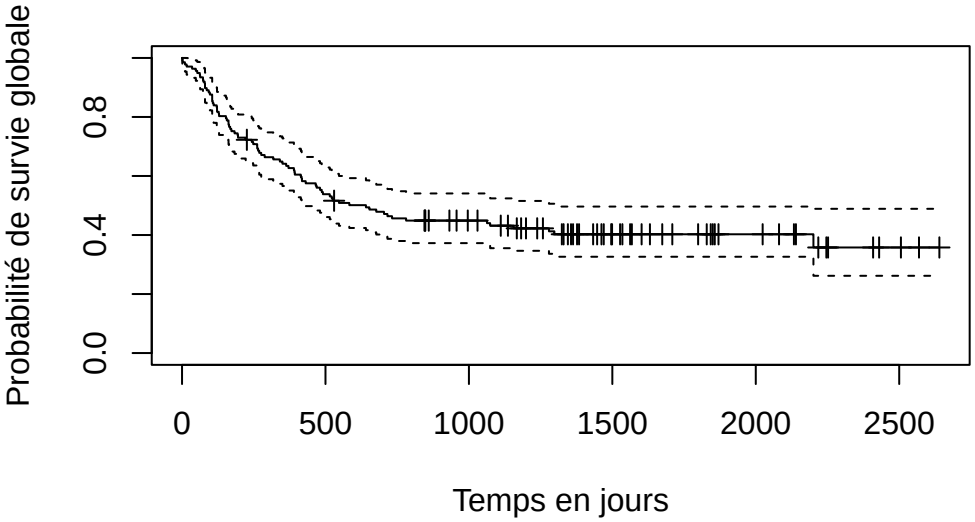


Le graphique obtenu est la représentation des estimations de Kaplan-Meïer. Le graphique par défaut manque d'information. En continuant avec les fonctions de base du package `survival`, il est possible de rajouter quelques informations, notamment les temps de censure à droite, et des labels aux axes.

Ajoutez les temps de censures (option `mark.time = TRUE`), et des labels informatifs aux axes (options `xlab = "mon label"` et `ylab = "mon label"`)

Correction :

```
plot(fit_km, mark.time = TRUE, xlab = "Temps en jours",
     ylab = "Probabilité de survie globale")
```



Une alternative "tout-en-un" existe, et utilise le package `ggsurvfit`. Il permet d'ajouter beaucoup d'informations utiles au graphique (médiane de survie, nombre d'individus exposés au risque, etc)

Le code ci-dessous n'a pas besoin d'être modifié. Conseil : utilisez le raccourci clavier Ctrl-Enter pour lancer ces commandes ligne après ligne et regarder le graphique produit à chaque fois (et non le bouton "Run Code" qui affichera tous les graphiques d'un seul coup).

```
library(ggsurvfit) ## Package à installer
p <- ggsurvfit(fit_km)
p ## Courbe de base

p + add_censor_mark() ## Ajout des censures
p + add_confidence_interval() ## Ajout de l'intervalle de confiance
p + add_quantile() ## Ajout de la médiane de survie
p + add_risktable() ## Ajout du nombre d'individus exposés au risque et du
                    ## nombre d'évènement
p + add_risktable(risktable_stats = "n.risk") ## Ajout du nombre d'individus
                                              ## exposés au risque uniquement
p + add_risktable(risktable_stats = "n.risk", times = seq(0, 2500, 500))
                                              ## Ajout du nombre d'individus
```

```
                                                    ## exposés au risque uniquement,
                                                    ## tous les 500 jours
p + scale_x_continuous(breaks = seq(0, 2500, 500)) ## Modification des temps
                                                         ## représentés
```

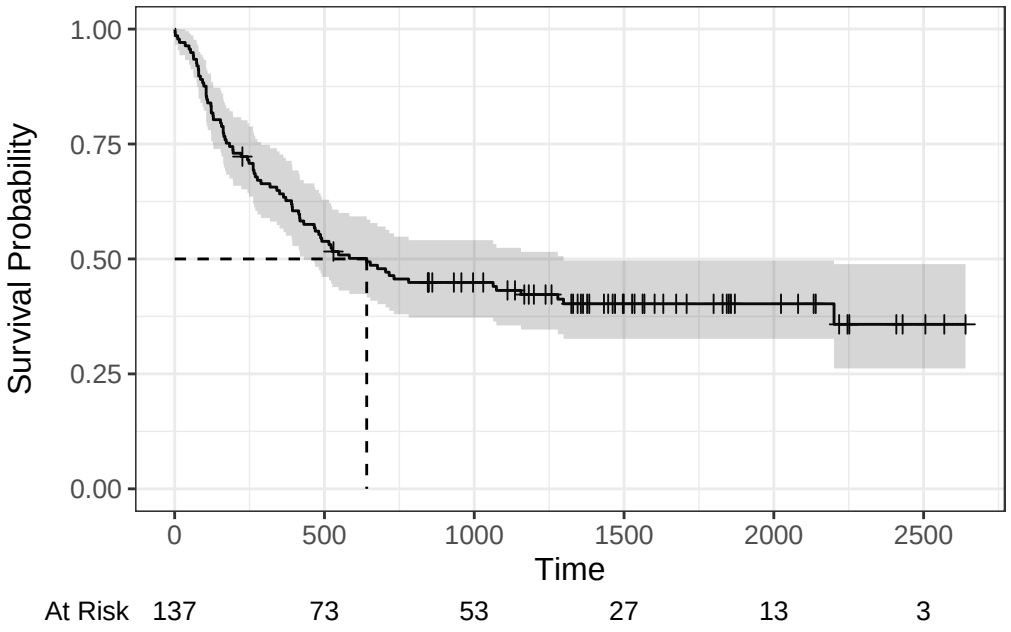
En vous basant sur le code ci-dessus qui explique chaque commande individuelle, construisez un graphique montrant à la fois les censures, l'intervalle de confiance, la médiane et le nombre d'individus exposés au risque

```
p <- ggsurvfit(fit_km) + ...
```

Correction :

```
p + add_censor_mark() +
  add_confidence_interval() +
  add_quantile() +
  add_risktable(risktable_stats = "n.risk", times = seq(0, 2500, 500)) +
  scale_x_continuous(breaks = seq(0, 2500, 500))

## Tips : seq(0, 2500, 500) : cette commande génère
## les nombres 0, 500, 1000, ..., 2500
```



2.3 Indicateurs

Pour évaluer la qualité du suivi dans l'étude, on peut calculer quelques indicateurs importants.

2.3.1 Médiane de survie

Nous avons déjà représenté la médiane de survie graphiquement. Son estimation numérique est directement affichée par l'objet créé par `survfit()` :

```
fit_km
```

2.3.2 Suivi médian

Le suivi médian demande un peu plus de travail ;-)

Pour l'obtenir, il faut de nouveau utiliser la fonction `survfit()` **en inversant le rôle de l'évènement et de la censure**.

Estimer le suivi médian

Correction :

```
survfit(Surv(delsv, 1-etat) ~ 1, data = bone)
```

```
Call: survfit(formula = Surv(delsv, 1 - etat) ~ 1, data = bone)

       n events median 0.95LCL 0.95UCL
[1,] 137     56   1499    1384    1709
```

En plus du suivi médian, on rapporte aussi souvent les *bornes du suivi*, c'est-à-dire :

- la durée observée de suivi **maximale**
- la durée observée de suivi **minimale des patients vivants**

Estimer les bornes du suivi (on utilisera notamment les fonctions `max()` et `min()`)

Correction :

```
max(bone$delsv)
min(bone$delsv[bone$etat == 0])

## Même chose avec dplyr
# library(dplyr)
bone %>% summarize(max(delsv))
bone %>% filter(etat == 0) %>% summarize(min(delsv))
```

```
Time difference of 2640 days
```

```
Time difference of 226 days
```

```
  max(delsv)
1  2640 days
```

```
  min(delsv)
1   226 days
```

Bravo, vous avez terminé cette partie du TP !

Pour passer à la suite, appuyez sur le lien en bas à droite de cette page (ou sur la partie suivante à gauche) !

3 Comparaison de la survie de plusieurs groupes

Si vous faites le TD avec Rstudio, la base de données doit avoir été importée avec les commandes suivantes :

```
bone <- read.csv2("chemin/vers/data_bone.csv")
bone$dat_sel <- as.Date(bone$dat_sel, format = "%d/%m/%Y")
bone$date_dc <- as.Date(bone$date_dc, format = "%d/%m/%Y")
bone$ddn <- as.Date(bone$ddn, format = "%d/%m/%Y")
bone$date_rec <- as.Date(bone$date_rec, format = "%d/%m/%Y")
bone$date_fn <- as.Date(bone$date_fn, format = "%d/%m/%Y")
bone$date_cc <- as.Date(bone$date_cc, format = "%d/%m/%Y")
bone$date_ca <- as.Date(bone$date_ca, format = "%d/%m/%Y")
```

Les variables suivantes doivent également avoir été créées.

```
bone$delsv <- difftime(bone$date_dc, bone$dat_sel,units = "days")
bone$delsv <- ifelse(bone$etat == 1, bone$delsv,
                     difftime(bone$ddn, bone$dat_sel, units = "days"))
bone$leuc.f <- factor(bone$leuc, 1:3, c("LAM B", "LAM C", "LAL"))
```

3.1 Survie globale par groupe

Les variables `delsv` et `etat` créées dans la partie précédente nous ont permis d'estimer puis représenter la survie globale avec la méthode de Kaplan-Meïer. Pour cela, nous avons utilisé la fonction `survfit()` (qui exécute tous les calculs), puis les fonctions `summary()`, `plot()` et `ggsurvfit` (qui permettent d'extraire les résultats souhaités).

Pour le moment, nous avons estimé la survie globale dans l'ensemble de l'échantillon. Mais il est souvent intéressant de l'estimer selon des **groupes définis au début du suivi**.

Entre la partie précédente du TP et cette partie, une nouvelle variable est apparue dans la data.frame `bone`. Il s'agit de la version `factor` de la variable catégorielle `leuc`, codée numériquement jusque là. Son nom se termine par `.f` :

```
## leuc.f a été préalablement générée avec la commande suivante :
## bone$leuc.f <- factor(bone$leuc, 1:3, c("LAM B", "LAM C", "LAL"))

table(bone$leuc, bone$leuc.f)
```

Vous voyez que chaque valeur de la variable `leuc` correspond à un niveau de la variable `leuc.f`.

Nous pouvons maintenant estimer la **survie globale par niveau de la variable `leuc.f`** ! Pour cela, c'est à nouveau la fonction `survfit()` qui entre en jeu, comme dans la partie précédente du TP. Nous allons juste changer la formule.

Estimez la survie globale selon la variable `leuc.f`. Pour indiquer à R que l'on souhaite faire une estimation de Kaplan-Meïer par groupe, il faut remplacer `~ 1` par `~ nomvariable` dans la formule utilisée avec la commande `survfit`. Vous stockerez le résultat dans un objet nommé `fit_km_leuc` avant d'afficher le résultat.

Correction :

```
fit_km_leuc <- survfit(Surv(delsv, etat) ~ leuc.f, data = bone)
fit_km_leuc
```

```
Call: survfit(formula = Surv(delsv, etat) ~ leuc.f, data = bone)

              n events median 0.95LCL 0.95UCL
leuc.f=LAM B 38     24    466     276      NA
leuc.f=LAM C 54     23   2201    1074      NA
leuc.f=LAL   45     34    265     162     547
```

Comme précédemment, utilisez les fonctions `summary`, `plot` et `ggsurvfit`.

Correction :

```
summary(fit_km_leuc, times = c(365*0:7))
plot(fit_km_leuc)
ggsurvfit(fit_km_leuc)
```

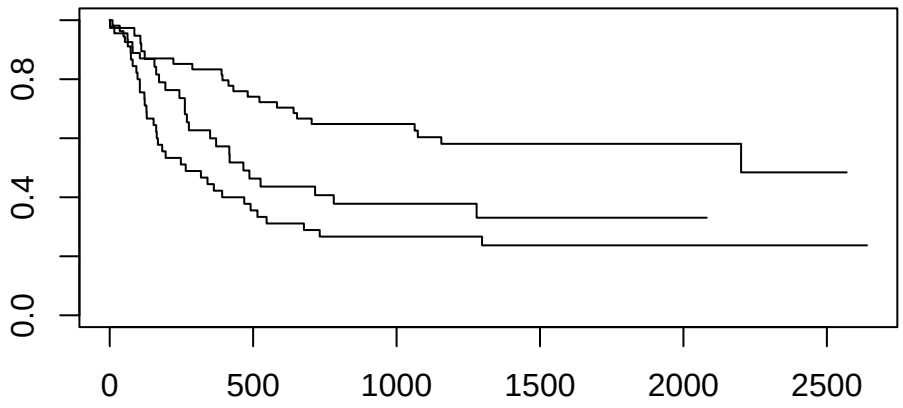
```
Call: survfit(formula = Surv(delsv, etat) ~ leuc.f, data = bone)

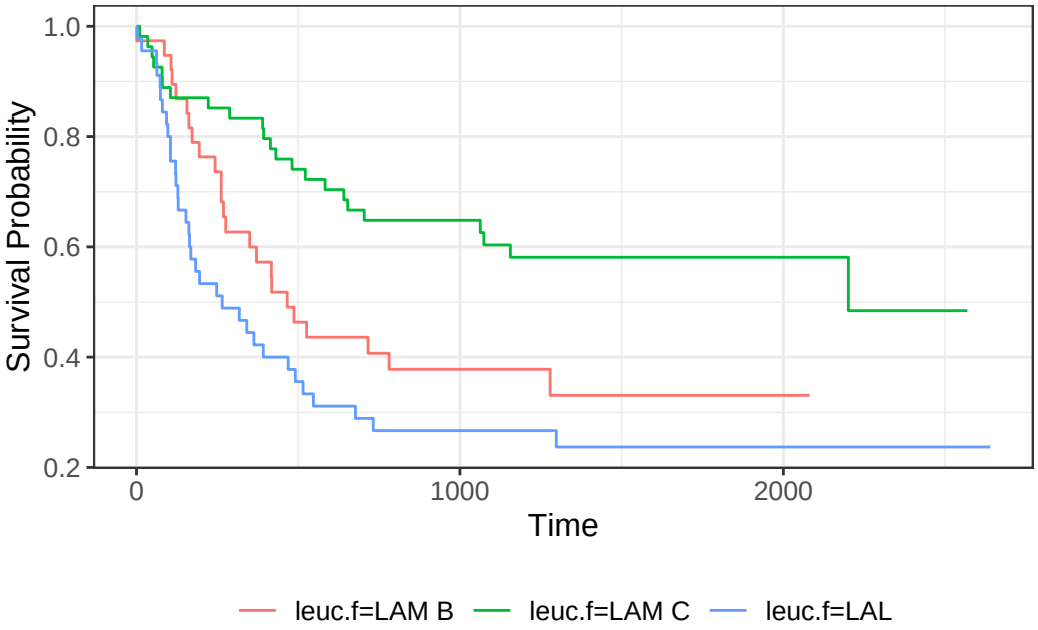
                leuc.f=LAM B
 time n.risk n.event survival std.err lower 95% CI upper 95% CI
    0     38       0    1.000  0.0000        1.000        1.000
  365     22      15    0.600  0.0802        0.461        0.779
  730     14       7    0.407  0.0811        0.275        0.602
 1095     12       1    0.378  0.0804        0.249        0.573
 1460      4       1    0.331  0.0830        0.202        0.541
 1825      1       0    0.331  0.0830        0.202        0.541

                leuc.f=LAM C
```

```
time n.risk n.event survival std.err lower 95% CI upper 95% CI
   0     54       0    1.000  0.0000        1.000        1.000
 365     45       9    0.833  0.0507        0.740        0.939
 730     35      10    0.648  0.0650        0.533        0.789
1095     27       2    0.603  0.0678        0.484        0.752
1460     20       1    0.581  0.0688        0.461        0.733
1825     11       0    0.581  0.0688        0.461        0.733
2190      6       0    0.581  0.0688        0.461        0.733
2555      1       1    0.484  0.1054        0.316        0.742

               leuc.f=LAL
time n.risk n.event survival std.err lower 95% CI upper 95% CI
   0     45       0    1.000  0.0000        1.000        1.000
 365     19      26    0.422  0.0736        0.300        0.594
 730     13       6    0.289  0.0676        0.183        0.457
1095     11       1    0.267  0.0659        0.164        0.433
1460      7       1    0.237  0.0649        0.139        0.405
1825      6       0    0.237  0.0649        0.139        0.405
2190      3       0    0.237  0.0649        0.139        0.405
2555      1       0    0.237  0.0649        0.139        0.405
```





L'affichage par défaut de la fonction `plot()` ne permet pas de distinguer quelle courbe correspond à quel groupe... Même s'il est possible de remédier à cela avec les fonctions de base du package `survival`, la fonction `ggsurvfit()` du package du même nom est bien plus facile à utiliser. Le graphique obtenu est encore plus propre si l'estimation de Kaplan-Meïer est obtenue via la fonction `survfit2()` au lieu de `survfit()`. Les résultats seront exactement les mêmes ! Mais la légende du graphique n'affichera pas le nom de la variable `leuc.f`. Ceci est uniquement cosmétique.

```
fit_km_leuc2 <- survfit2(Surv(delsv, etat) ~ leuc.f, data = bone)
ggsurvfit(fit_km_leuc2)
```

3.2 Comparaison de la survie globale

Pour comparer plusieurs courbes de survie, on peut utiliser **le test du logrank**. Son utilisation est simple : au lieu d'utiliser la fonction `survfit()` (pour estimer les probabilités de survie), on va utiliser la fonction `survdiff()`. La syntaxe entre ces deux fonctions est la même ! Il suffit donc de remplacer `survfit` par `survdiff`.

Réalisez un test du logrank pour comparer la survie globale entre les groupes définis par la variable ***leuc.f****.*

Correction :

```
survdiff(Surv(delsv, etat) ~ leuc.f, data = bone)
```

```
Call:
survdiff(formula = Surv(delsv, etat) ~ leuc.f, data = bone)
```

```
               N Observed Expected (O-E)^2/E (O-E)^2/V
leuc.f=LAM B 38       24     21.3     0.337     0.462
leuc.f=LAM C 54       23     38.8     6.413    12.562
leuc.f=LAL   45       34     20.9     8.190    11.162

 Chisq= 15.2  on 2 degrees of freedom, p= 5e-04
```

Le résultat affiché vous donne :

- les divers ingrédients de la statistique de test (effectifs, nombre d'évènements observés et attendus, etc)
- la statistique de test (`Chisq=...`)
- le nombre de degrés de liberté du test (soit le nombre de groupes comparés moins un)
- le degré de signification

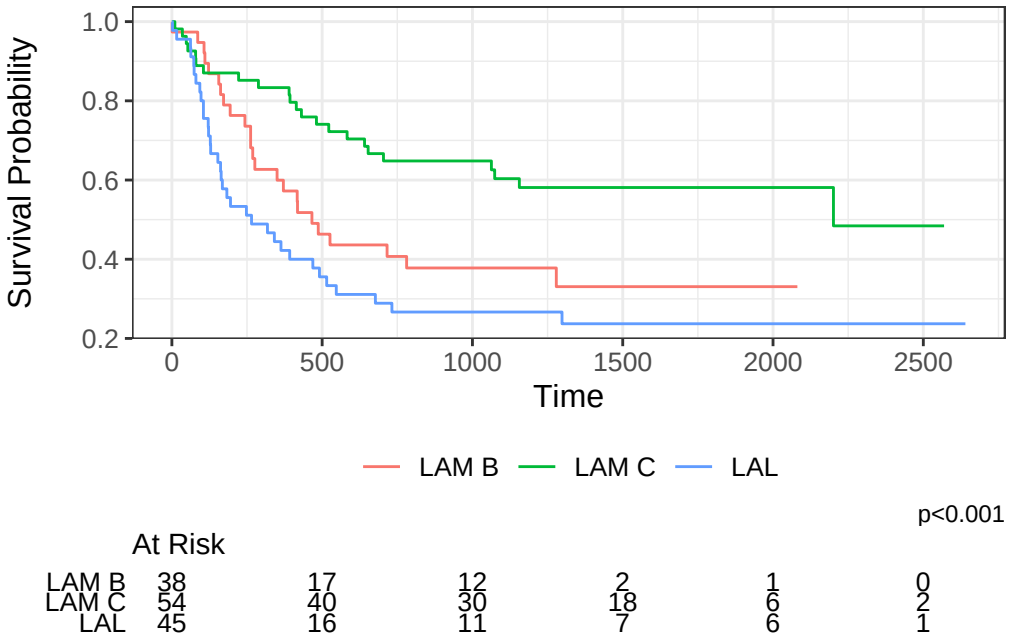
Il est possible d'afficher le résultat du test du logrank sur le graphique de Kaplan-Meïer. Là encore, c'est avec la fonction `ggsurvfit()` que le résultat souhaité est obtenu le plus facilement.

Affichez le résultat du test du logrank sur le graphique de Kaplan-Meïer. Vous utiliserez à nouveau la fonction ***`ggsurvfit()`****, en y ajoutant la commande* ***`+ add_pvalue()`****. Profitez-en pour ajouter également le nombre d'individus à risque.*

Correction :

```r
ggsurvfit(fit_km_leuc2) +
  add_pvalue() +
  add_risktable(risktable_stats = "n.risk", times = seq(0, 2500, 500)) +
  scale_x_continuous(breaks = seq(0, 2500, 500))

## Tips : seq(0, 2500, 500) : cette commande génère
## les nombres 0, 500, 1000, ..., 2500
```



Bravo, vous avez terminé cette partie du TP !

Pour passer à la suite, appuyez sur le lien en bas à droite de cette page (ou sur la partie suivante à gauche) !

4 Modèle de Cox

Si vous faites le TD avec Rstudio, la base de données doit avoir été importée avec les commandes suivantes :

```
bone <- read.csv2("chemin/vers/data_bone.csv")
bone$dat_sel <- as.Date(bone$dat_sel, format = "%d/%m/%Y")
bone$date_dc <- as.Date(bone$date_dc, format = "%d/%m/%Y")
bone$ddn <- as.Date(bone$ddn, format = "%d/%m/%Y")
bone$date_rec <- as.Date(bone$date_rec, format = "%d/%m/%Y")
bone$date_fn <- as.Date(bone$date_fn, format = "%d/%m/%Y")
bone$date_cc <- as.Date(bone$date_cc, format = "%d/%m/%Y")
bone$date_ca <- as.Date(bone$date_ca, format = "%d/%m/%Y")
```

Les variables suivantes doivent également avoir été créées.

```
bone$delsv <- difftime(bone$date_dc, bone$dat_sel,units = "days")
bone$delsv <- ifelse(bone$etat == 1, bone$delsv,
                     difftime(bone$ddn, bone$dat_sel,units = "days"))
bone$leuc.f <- factor(bone$leuc, 1:3, c("LAM B", "LAM C", "LAL"))
bone$mtx.f <- factor(bone$mtx, 0:1, c("Pas de MTX", "MTX"))
```

4.1 Modèle de Cox univariable

Après avoir comparé des courbes au moyen d'un test du logrank, il est temps maintenant de fitter notre premier modèle de Cox. Après avoir utilisé `survfit()`, puis `survdiff()`, vous allez maintenant utiliser la fonction `coxph()`. Là encore, la syntaxe reste la même !

Fittez un modèle de Cox évaluant l'effet de la variable ***`leuc.f`*** *sur la survie globale, à l'aide de la fonction* ***`coxph()`****. Vous stockerez le résultat dans un objet appelé* ***`fit_cox_leuc`*** *avant d'afficher le résultat.*

Correction :

```
fit_cox_leuc <- coxph(Surv(delsv, etat) ~ leuc.f, data = bone)
fit_cox_leuc
```

```
Call:
coxph(formula = Surv(delsv, etat) ~ leuc.f, data = bone)

               coef exp(coef) se(coef)      z      p
leuc.fLAM C -0.6552    0.5193   0.2937 -2.231 0.0257
leuc.fLAL    0.3702    1.4480   0.2674  1.384 0.1662

Likelihood ratio test=15  on 2 df, p=0.0005539
n= 137, number of events= 81
```

Le résultat affiché comporte déjà pas mal d'informations intéressantes :

- les coefficients associés aux catégories `LAM C` et `LAL`.
- l'exponentiel de ces coefficients
- l'écart-type de ces coefficients
- les p-values associées à ces coefficients
- la p-value associée au 'Likelihood ratio test'

Pourquoi aucun résultat n'est-il affiché pour la catégorie `LAM B` ? [1]

—(A) Cette catégorie ne comporte aucun patient.

—(B) Cette catégorie ne comporte aucun évènement.

—(C) Cette catégorie n'a aucun intérêt, c'est bien connu.

—(D) Cette catégorie est la catégorie de référence.

—(E) 42

Que représente l'exponentiel de chaque coefficient ? [2]

—(A) Une probabilité d'évènement.

—(B) Un odds ratio

—(C) Un risque relatif

—(D) Une différence de risque.

—(E) Un rapport de risque instantané (hazard ratio).

A quoi peut bien servir l'écart-type des coefficients ? [3]

—(A) à calculer un intervalle de pari

—(B) à calculer un intervalle de confiance et une statistique de test

1. Cette catégorie est la catégorie de référence.
2. Un rapport de risque instantané (hazard ratio).
3. à calculer un intervalle de confiance et une statistique de test

—(C) à mesurer la force de l'association

—(D) à rien

Comment s'appelle le test affiché pour chaque coefficient ? [4]

—(A) Le test du score.

—(B) Le test du logrank

—(C) Le test de Wald

—(D) Le test du χ^2.

—(E) Le test du rapport de vraisemblance.

Quelle(s) hypothèse(s) nulle(s) teste-t-il ? (: la valeur théorique du coefficient concerné) [5]

—(A) $H_0 : \beta = 0$

—(B) $H_0 : \beta = 1$

—(C) $H_0 : \beta < 0$

—(D) $H_0 : \beta > 0$

Même question pour le 'Likelihood ratio test' affiché ? (et : la valeur théorique de chaque coefficient) [6]

—(A) $H_0 : \beta_1 = 0$

—(B) $H_0 : \beta_2 = 0$

—(C) $H_0 : \beta_1$ ou $\beta_2 = 0$

—(D) $H_0 : \beta_1$ et $\beta_2 = 0$

Comme pour les objets produits par `survfit()`, il est possible d'enrichir la sortie par défaut, à l'aide de la fonction `summary()`.

Appliquez `summary()` à l'objet `fit_cox_leuc`.

Correction :

```
summary(fit_cox_leuc)
```

---

4. Le test de Wald
5. $H_0 : \beta = 0$
6. $H_0 : \beta_1$ et $\beta_2 = 0$

```
Call:
coxph(formula = Surv(delsv, etat) ~ leuc.f, data = bone)

  n= 137, number of events= 81

               coef exp(coef) se(coef)      z Pr(>|z|)
leuc.fLAM C -0.6552    0.5193   0.2937 -2.231   0.0257 *
leuc.fLAL    0.3702    1.4480   0.2674  1.384   0.1662
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

            exp(coef) exp(-coef) lower .95 upper .95
leuc.fLAM C    0.5193     1.9256    0.2921    0.9235
leuc.fLAL      1.4480     0.6906    0.8573    2.4457

Concordance= 0.628  (se = 0.031 )
Likelihood ratio test= 15  on 2 df,   p=6e-04
Wald test            = 14.25  on 2 df,   p=8e-04
Score (logrank) test = 15.22  on 2 df,   p=5e-04
```

Le résultat s'est enrichi. On remarque notamment les bornes de l'intervalle de confiance à 95% autour des coefficients (`lower .95` et `upper .95`)

4.2 Hypothèse des hasards proportionnels

Les hazard ratios estimés par le modèle de Cox ne sont réellement interprétables que sous l'hypothèse des hazards proportionnels. Il existe un test (H_0 : les hazards sont proportionnels) qui permet de vérifier que cette hypothèse n'est pas significativement rejetée (on l'admettra 'vraie' dans ce cas) : le **test de Harrell**. Il s'obtient de la manière suivante :

Lancer la ligne de code suivante.

```
cox.zph(fit_cox_leuc, transform = "rank", terms = FALSE)
```

On obtient un tableau une ligne par coefficient estimé + un test global. La colonne `p` indique le degré de signification du test. Ici, l'hypothèse des hasards proportionnels n'est pas rejetée, pour les deux coefficients estimés : on n'a pas mis en évidence d'écart à l'hypothèse des hasards proportionnels.

Le résultat de la fonction `cox.zph` peut également être représenté graphiquement. La ligne obtenue (courbe lissée passant au milieu des *résidus de Schoenfeld*, entourée de son intervalle de confiance à 95%) est une estimation de l'effet dépendant du temps de la variable explicative. Si l'hypothèse des hazards proportionnels est valide, alors la ligne obtenue devrait être globalement horizontale (en pratique, on regarde si une droite horizontale peut être tracée sans déborder de l'intervalle de confiance à 95%). Voici ce qu'on obtient pour la variable `leuc.f` (remarquez que pour obtenir ce graphique, on n'utilise pas l'option `transform = "rank"`, mais `transform = "identity"`) :

Lancer la ligne de code suivante.

```
plot(cox.zph(fit_cox_leuc, transform = "identity", terms = FALSE))
```

Fittez un modèle de Cox évaluant l'effet de la variable ***mtx.f*** *sur la survie globale, stockez le résultat dans un objet nommé* ***fit_cox_mtx****, puis évaluez l'hypothèse des hasards proportionnels.*

Correction :

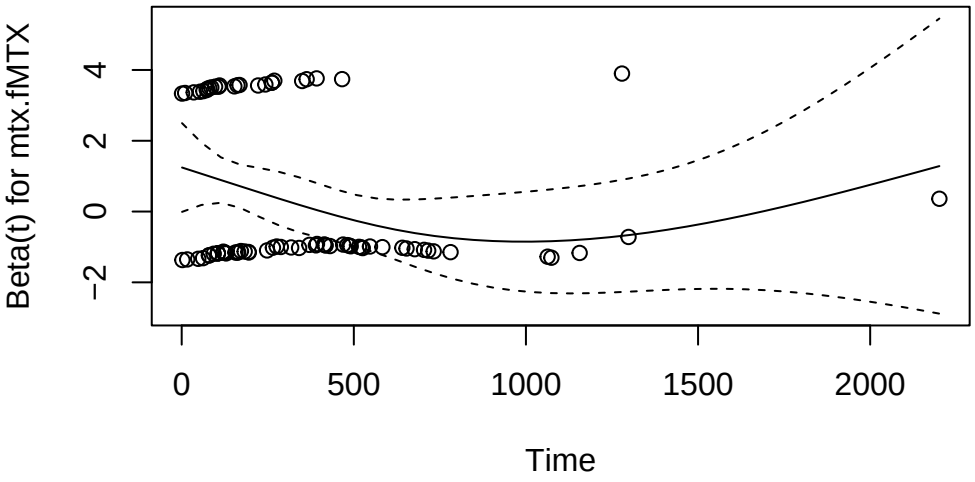
```
## mtx.f a été préalablement générée avec la commande suivante :
# bone$mtx.f <- factor(bone$mtx, 0:1, c("Pas de MTX", "MTX"))
fit_cox_mtx <- coxph(Surv(delsv, etat) ~ mtx.f, data = bone)
fit_cox_mtx
cox.zph(fit_cox_mtx, transform = "rank", terms = FALSE)
plot(cox.zph(fit_cox_mtx, transform = "identity", terms = FALSE))
```

```
Call:
coxph(formula = Surv(delsv, etat) ~ mtx.f, data = bone)

           coef exp(coef) se(coef)     z     p
mtx.fMTX 0.3614    1.4353   0.2417 1.495 0.135

Likelihood ratio test=2.13  on 1 df, p=0.1445
n= 137, number of events= 81
```

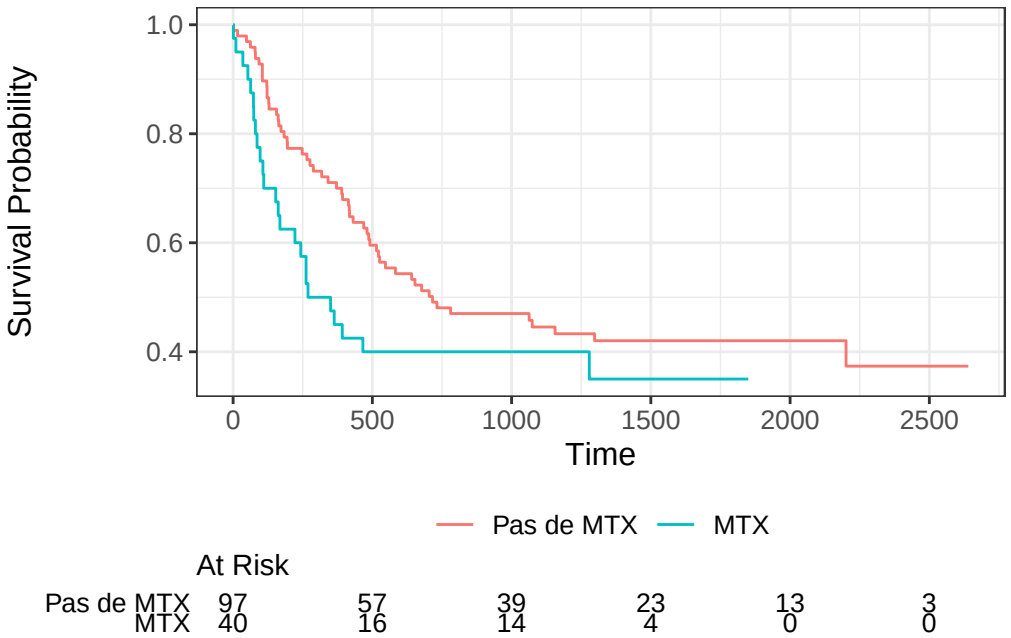
```
         chisq df     p
mtx.fMTX  6.25  1 0.012
GLOBAL    6.25  1 0.012
```



Tracez maintenant la survie globale en fonction des catégories de la variable ***mtx.f****. Pouvait-on s'attendre au résultat du test de Harrell ? Pourquoi ?*

Correction :

```r
fit_km_mtx <- survfit2(Surv(delsv, etat) ~ mtx.f, data = bone)
ggsurvfit(fit_km_mtx) +
  scale_x_continuous(breaks = seq(0, 2500, 500)) +
  add_risktable(risktable_stats = "n.risk", times = seq(0, 2500, 500))
```



Il existe plusieurs approches en cas d'écart à l'hypothèse des hazards proportionnels, chacune ayant des objectifs, avantages et inconvénients différents. On distingue notamment :

— le modèle de Cox stratifié,
— le test du logrank pondéré,
— la modélisation d'une variable dont l'effet dépend du temps.

Ces approches sortent du cadre de cette introduction aux analyses de survie.

4.3 Modèle de Cox multivariable

Pour fitter un modèle de Cox incluant plusieurs variables explicatives, il suffit de les ajouter à la formule précédente, séparées du signe +.

Fittez un modèle de Cox évaluant l'effet des variables `leuc.f` *ET* `age.pat` *sur la survie globale, et stockez le résultat dans un objet nommé* `fit_cox_leuc_age` *avant d'afficher le résultat détaillé.*

Correction :

```r
fit_cox_leuc_age <- coxph(Surv(delsv, etat) ~ leuc.f + age.pat, data = bone)
summary(fit_cox_leuc_age)
```

```
Call:
coxph(formula = Surv(delsv, etat) ~ leuc.f + age.pat, data = bone)

  n= 137, number of events= 81

                coef exp(coef) se(coef)      z Pr(>|z|)
leuc.fLAM C -0.72009   0.48671  0.30265 -2.379   0.0173 *
leuc.fLAL    0.28125   1.32479  0.28596  0.984   0.3253
age.pat      0.01138   1.01144  0.01224  0.929   0.3527
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

            exp(coef) exp(-coef) lower .95 upper .95
leuc.fLAM C    0.4867     2.0546    0.2689    0.8808
leuc.fLAL      1.3248     0.7548    0.7564    2.3203
age.pat        1.0114     0.9887    0.9875    1.0360

Concordance= 0.648  (se = 0.032 )
Likelihood ratio test= 15.86  on 3 df,   p=0.001
Wald test            = 15.18  on 3 df,   p=0.002
Score (logrank) test = 16.15  on 3 df,   p=0.001
```

Comment s'interprète la valeur du HR associé à l'âge du patient ? [7]

—(A) Les patients âgés ont 1.01 fois plus de risque de décès que les patients jeunes.

—(B) Les patients jeunes ont 1.01 fois plus de risque de décès que les patients âgés.

—(C) Le risque de décès est multiplié par 1.01 à chaque incrément d'un an de l'âge du patient.

—(D) Le risque de décès est multiplié par 1.01 à chaque année de suivi en plus.

Le résultat détaillé de ce modèle multivarié fournit :

— les tests de Wald associés à chaque coefficient ($H_0 : \beta_k = 0$)
— les tests globaux (Wald, score et rapport de vraisemblance, $H_0 : \beta_1 = \beta_2 = \beta_3 = 0$)

Pour tester la significativité de la variable `leuc.f` *dans son ensemble* dans ce modèle multivariable, on peut utiliser le test du rapport de vraisemblance. Pour l'obtenir, il faut comparer deux modèles avec la fonction `anova()` : le modèle avec `leuc.f + age.pat`, et le modèle avec `age.pat` seulement.

Testez la variable `leuc.f` à l'aide du test du rapport de vraisemblance.

Correction :

```r
fit_cox_age <- coxph(Surv(delsv, etat) ~ age.pat, data = bone)
anova(fit_cox_leuc_age, fit_cox_age)
```

7. Le risque de décès est multiplié par 1.01 à chaque incrément d'un an de l'âge du patient.

```
Analysis of Deviance Table
 Cox model: response is  Surv(delsv, etat)
 Model 1: ~ leuc.f + age.pat
 Model 2: ~ age.pat
   loglik  Chisq Df Pr(>|Chi|)
1 -356.42
2 -363.82 14.804  2    0.00061 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Bravo, vous avez terminé cette partie du TP !

Pour passer à la suite, appuyez sur le lien en bas à droite de cette page (ou sur la partie suivante à gauche) !

5 Risques compétitifs

Si vous faites le TD avec Rstudio, la base de données doit avoir été importée avec les commandes suivantes :

```
bone <- read.csv2("chemin/vers/data_bone.csv")
bone$dat_sel <- as.Date(bone$dat_sel, format = "%d/%m/%Y")
bone$date_dc <- as.Date(bone$date_dc, format = "%d/%m/%Y")
bone$ddn <- as.Date(bone$ddn, format = "%d/%m/%Y")
bone$date_rec <- as.Date(bone$date_rec, format = "%d/%m/%Y")
bone$date_fn <- as.Date(bone$date_fn, format = "%d/%m/%Y")
bone$date_cc <- as.Date(bone$date_cc, format = "%d/%m/%Y")
bone$date_ca <- as.Date(bone$date_ca, format = "%d/%m/%Y")
```

Les variables suivantes doivent également avoir été créées.

```
bone$delsv <- difftime(bone$date_dc, bone$dat_sel,units = "days")
bone$delsv <- ifelse(bone$etat == 1, bone$delsv,
                     difftime(bone$ddn, bone$dat_sel, units = "days"))
bone$leuc.f <- factor(bone$leuc, 1:3, c("LAM B", "LAM C", "LAL"))
bone$mtx.f <- factor(bone$mtx, 0:1, c("Pas de MTX", "MTX"))
bone$fab.f <- factor(bone$fab, 0:1, c("Grade 4 ou 5 ou LMA ", "Autres"))
bone$delrec <- difftime(bone$date_rec, bone$dat_sel, units = "days")
bone$delrec <- ifelse(bone$rec == 1, bone$delrec,
                      difftime(bone$ddn, bone$dat_sel, units = "days"))
```

5.1 Délai jusqu'à récidive, survie sans récidive

La data.frame `bone` contient maintenant un nouveau délai jusqu'à évènement, au travers des colonnes `delrec` et `rec` :

```
head(bone[, c("delrec", "rec")])
summary(bone[, c("delrec", "rec")])
```

Le délai jusqu'à récidive a été construit comme suit :

- si un patient récidive (qu'il meurt ou non après) : `delrec` = délai jusqu'à récidive, `rec` = 1
- si un patient meurt sans récidive : `delrec` = délai jusqu'au décès, `rec` = 0
- si un patient est censuré vivant sans récidive : `delrec` = délai jusqu'aux dernières nouvelles, `rec` = 0

(Pour information, la variable `delrec` a été créée avec le code suivant).

```r
bone$delrec <- difftime(bone$date_rec, bone$dat_sel, units = "days")
bone$delrec <- ifelse(bone$rec == 1,
                      bone$delrec,
                      difftime(bone$ddn, bone$dat_sel, units = "days"))
```

Pour des raisons évidentes, la récidive est en compétition avec le décès. Une première approche possible est de créer un critère composite : le délai jusqu'à récidive OU décès (souvent appelé 'survie sans récidive' ou 'disease-free survival' en anglais).

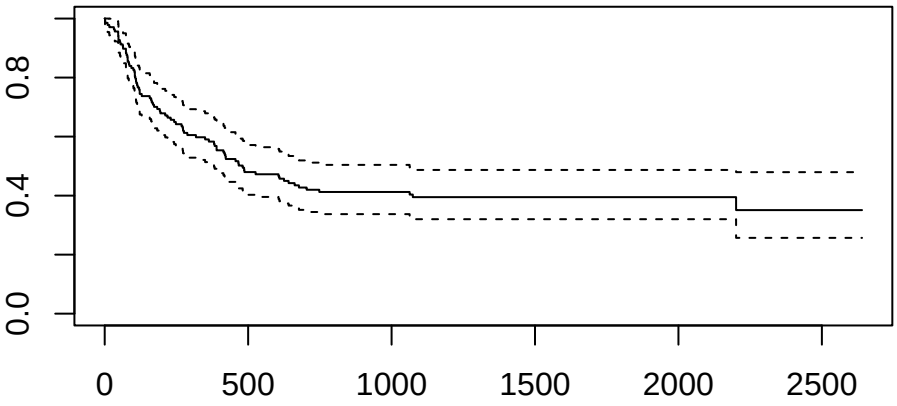
Créez le critère composite 'délai jusqu'à récidive OU décès sans récidive', puis décrivez le avec la méthode de Kaplan-Meïer. La fonction ***pmin()*** *pourra être très utile pour calculer le délai jusqu'au premier des deux évènements.*

Correction :

```r
## Calcul du délai jusqu'au premier évènement
bone$deldfs <- pmin(bone$delrec, bone$delsv) ## pour chaque patient, délai
                                             ## minimal entre delrec et delsv
### Alternative : apply(bone[, c("delrec", "delsv")], 1, min)

## Evènement : récidive OU décès
bone$dfs <- ifelse(bone$rec == 1 | bone$etat == 1 , 1, 0)
plot(survfit(Surv(deldfs, dfs) ~ 1, data = bone)) ## Fonction de survie
                                                  ## (comme d'habitude)

## Même chose avec dplyr
# library(dplyr)
bone <- bone %>%
    rowwise() %>%
    mutate(
      deldfs = min(delrec, delsv),
      dfs = case_when(
        rec == 1 | etat == 1 ~ 1,
        rec == 0 & etat == 0 ~ 0
      )
    )
```



Avec cette approche, il n'y a plus de risques en compétition : c'est l'évènement d'intérêt qui est redéfini et qui devient unique (mais composite).

Jusqu'ici, nous avons à chaque fois représenté la fonction de survie $S(t)$. Mais il est très facile de représenter son complémentaire, la fonction d'incidence cumulée, en ajoutant l'option `fun = "F"` :

```
## Fonction d'incidence cumulée (F(t) = 1-S(t))
plot(survfit(Surv(deldfs, dfs) ~ 1, data = bone), fun = "F")
```

5.2 Délai jusqu'à récidive ou décès sans récidive en risques compétitifs

En risques compétitifs, la variable de délai `deldfs` est inchangée, mais les deux types d'évènement (récidive, ou décès sans récidive) doivent rester séparés. Nous devons créer une colonne de type `factor`, avec trois niveaux :

- 'censor' : si le patient est censuré sans récidive ni décès (`etat` $= 0$ ET `rec` $= 0$)
- 'relapse' : si le patient récidive avec ou sans décès par la suite (`rec` $= 1$)
- 'death without relapse' : si le patient décède sans récidive (`etat` $= 1$ ET `rec` $= 0$)

Créez la variable adéquate. Vous stockerez le résultat dans une nouvelle colonne de la data.frame ***bone****, appelée* ***compevent****.*

Correction :

```
bone$compevent <- NA
bone$compevent[bone$etat == 0 & bone$rec == 0] <- 0
bone$compevent[bone$rec == 1] <- 1
bone$compevent[bone$etat == 1 & bone$rec == 0] <- 2
bone$compevent <- factor(bone$compevent, c(0, 1, 2),
                         c("censor", "relapse", "death without relapse"))

## Même chose avec dplyr
# library(dplyr)
bone <- bone %>%
  mutate(
    compevent = factor(case_when(
```

```
      etat == 0 & rec == 0 ~ "censor",
      rec == 1 ~ "relapse",
      etat == 1 & rec == 0 ~ "death without relapse"
    ))
  )
```

Il y a plusieurs manières de faire. Dans tous les cas, vous devez obtenir la répartition suivante :

```
table(bone$compevent, bone$etat)
```

```
                        0  1
  censor                54  0
  death without relapse  0 41
  relapse                2 40
```

Nous allons maintenant estimer l'incidence cumulée des deux évènements en compétition au cours du temps.

5.3 Incidence cumulée des évènements

5.3.1 Approche 'naïve'

Sans risque en compétition, l'incidence cumulée d'un évènement est le complémentaire de la fonction de survie ($F(t) = 1 - S(t)$). Avec le package `survival`, on estime de nouveau avec la fonction `survfit()`, puis il suffit de rajouter l'option `fun = "F"` à la fonction `plot()`. Ci-dessous, les fonctions d'incidence cumulée des évènements `"death without relapse"` (noir) et `"relapse"` (rouge) ont chacune été représentées, en censurant les patients ayant fait l'autre évènement. (Les intervalles de confiance ont été supprimés avec l'option `conf.int = FALSE` afin de rendre le graphique plus lisible).

```
plot(survfit(Surv(deldfs, compevent == "death without relapse") ~ 1, data = bone),
     fun = "F", conf.int = FALSE)
lines(survfit(Surv(deldfs, compevent == "relapse") ~ 1, data = bone),
      fun = "F", conf.int = FALSE, col = 2)
```

Cette approche est-elle correcte ? [1]

—(A) Oui, pas de soucis.

—(B) Non, car à l'infini, la probabilité de faire l'un des deux évènements est > 1.

1. Non, car à l'infini, la probabilité de faire l'un des deux évènements est > 1.

5.3.2 Approche correcte

Censurer les patients ayant fait l'évènement en compétition ne convient pas. Pour obtenir les fonctions d'incidence cumulée correctes, on peut utiliser la fonction `cuminc()` du package `tidycmprsk`. Elle s'utilise exactement de la même manière que la fonction `survfit()`. Le graphique correspondant peut être facilement obtenu avec la fonction `ggcuminc()` du package `ggsurvfit`.

Estimez les fonctions d'incidence cumulée des deux évènements en compétition à l'aide de la fonction ***`cuminc()`****. Vous stockerez le résultat dans un objet appelé* ***`fit_cuminc`****.*

```
library(tidycmprsk)
```

Correction :

```
library(tidycmprsk)
fit_cuminc <- cuminc(Surv(deldfs, compevent) ~ 1, data = bone)
fit_cuminc
```

```
-- cuminc() --------------------------------------------------------------------

* Failure type "death without relapse"

time    n.risk   estimate   std.error   95% CI
500     65       0.249      0.037       0.180, 0.324
1,000   48       0.279      0.039       0.206, 0.356
1,500   27       0.296      0.040       0.221, 0.375
2,000   13       0.296      0.040       0.221, 0.375
2,500   3        0.340      0.058       0.231, 0.453

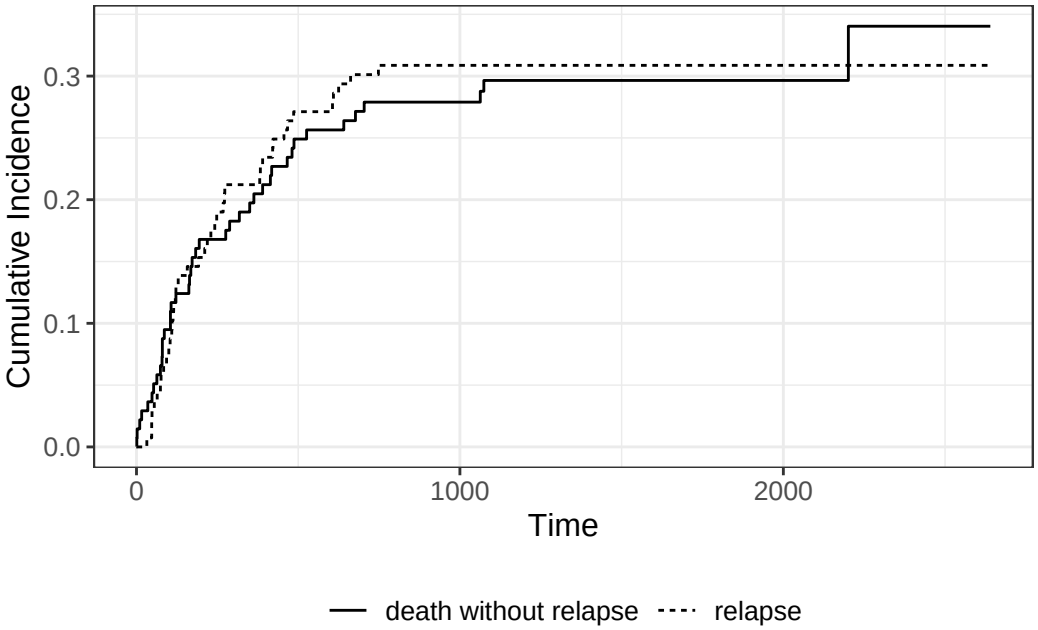
* Failure type "relapse"

time    n.risk   estimate   std.error   95% CI
500     65       0.271      0.038       0.199, 0.348
1,000   48       0.309      0.040       0.233, 0.388
1,500   27       0.309      0.040       0.233, 0.388
2,000   13       0.309      0.040       0.233, 0.388
2,500   3        0.309      0.040       0.233, 0.388
```

Tracez les fonctions d'incidence estimées à l'aide de la fonction `ggcuminc()`*. Par défaut, un seul évènement est représenté, mais vous pouvez ajouter l'option* `outcome = c("relapse", "death without relapse")` *pour obtenir les deux.*

Correction :

```
ggcuminc(fit_cuminc, outcome = c("relapse", "death without relapse"))
```



5.4 Modèle de Cox cause-spécifique

5.4.1 En deux commandes

Pour modéliser le risque de chacun des deux évènements en fonction des caractéristiques des patients, une première approche consiste à s'intéresser au risque cause-spécifique (appelé aussi 'intensité de transition'). Nous avons déjà utilisé la fonction `coxph()` du package `survival`… Et bien, nous allons l'utiliser à nouveau, car ce modèle s'obtient en *censurant les patients ayant fait l'autre évènement* (**attention, cela ne veut pas dire que cette manière de faire convient pour estimer la fonction d'incidence cumulée**).

Il faut donc estimer deux modèles de Cox : l'un en censurant les patients décédés sans rechute, l'autre en censurant les patients ayant rechuté avant un éventuel décès.

Estimez un modèle de Cox cause-spécifique pour chacun des deux évènements. Vous ne mettrez qu'une seule variable explicative dans la formule : `fab.f` *(classification des leucémies en deux catégories : 'Grade 4 ou 5 ou LMA' vs 'Autres').*

Correction :

```
coxph(Surv(deldfs, compevent == "relapse") ~ fab.f, data = bone)
coxph(Surv(deldfs, compevent == "death without relapse") ~ fab.f, data = bone)
```

```
Call:
coxph(formula = Surv(deldfs, compevent == "relapse") ~ fab.f,
    data = bone)

              coef exp(coef) se(coef)     z        p
fab.fAutres 1.0441    2.8408   0.3099 3.369 0.000755

Likelihood ratio test=11.05  on 1 df, p=0.0008877
n= 137, number of events= 42


Call:
coxph(formula = Surv(deldfs, compevent == "death without relapse") ~
    fab.f, data = bone)

              coef exp(coef) se(coef)     z     p
fab.fAutres 0.1893    1.2084   0.3369 0.562 0.574

Likelihood ratio test=0.31  on 1 df, p=0.5788
n= 137, number of events= 41
```

5.4.2 En une seule commande

Les deux modèles peuvent être estimés en une seule commande. Avec cette syntaxe, il est nécessaire d'ajouter l'option `id = num`, (`num` étant la colonne identifiant chaque individu) :

```
coxph(Surv(deldfs, compevent) ~ fab.f, data = bone, id = num)
```

Par défaut, cette approche calcule en plus une variance dite 'robuste' (`robust se`), c'est pourquoi les degrés de signification diffèrent légèrement (mais les coefficients estimés sont bien les mêmes que précédemment).

5.5 Modèle de Fine et Gray

Une seconde approche consiste à modéliser le risque de sous-répartition à l'aide d'un modèle de Fine et Gray. Techniquement parlant, la seule différence avec l'approche cause-spécifique concerne les individus à risque au cours du temps :

— Cox cause-spécifique : les patients ayant fait l'évènement en compétition sont censurés ⇒ ils ne sont plus dans le risk set après l'autre évènement

— Fine et Gray : les patients ayant fait l'évènement en compétition ne sont jamais censurés ⇒ ils restent dans le risk set après l'autre évènement

Estimer un modèle de Fine et Gray est donc très simple en réalité : il suffit d'attribuer aux individus ayant fait l'évènement en compétition un délai factice supérieur à tous les autres délais de suivi, afin qu'ils restent dans le risk set en permanence. Le modèle de Fine et Gray prend également en compte le fait que les censures peuvent elles-mêmes être masquées par l'évènement en compétition, par une approche de pondération des individus au cours du temps.

La fonction `crr()` (pour 'competing risk regression') du package `tidycmprsk` permet d'estimer un modèle de Fine et Gray.

```
crr(Surv(deldfs, compevent) ~ fab.f, data = bone, failcode = c("relapse"))
crr(Surv(deldfs, compevent) ~ fab.f, data = bone, failcode = c("death without relapse"))
```

5.6 Comparaison des modèles causes-spécifiques et de sous-répartition

Les deux approches conduisent à des résultats assez similaires.

On peut cependant remarquer que, pour l'évènement 'death without relapse', le *sub hazard ratio* estimé par le modèle de Fine et Gray est cohérent avec les incidences cumulées estimées :

```
ggcuminc(cuminc(Surv(deldfs, compevent) ~ fab.f, data = bone),
         outcome = c("relapse", "death without relapse")) +
  facet_grid(~ outcome)
```

Ainsi :

— l'incidence cumulée de 'death without relapse' chez les individus 'Autres' est légèrement plus faible que chez les individus 'Grade 4 ou 5 ou LMA'
— le *sub hazard ratio* estimé par le modèle de Fine et Gray est < 1 (sHR $= 0.92$) ⇒ cohérence avec ce qui est observé sur les incidences cumulées
— le *hazard ratio cause-spécifique* estimé par le modèle de Cox cause-spécifique est > 1 (csHR $= 1.21$) ⇒ pas de cohérence avec ce qui est observé sur les incidences cumulées

Pour passer à la suite, appuyez sur le lien en bas à droite de cette page (ou sur la partie suivante à gauche) !

Conclusion

Bravo, vous avez terminé ce TP dédié aux analyses de survie !

La base de données 'bone' contient d'autres variables et d'autres types d'évènements que ceux vus durant ce TP, n'hésitez pas à continuer de l'explorer (la base de données est téléchargeable depuis la page d'accueil de ce TP).